Ho Chi Minh City University of Technology (HCMUT), VNU-HCM

Faculty of Computer Science and Engineering



ASSIGNMENT 1

Foundations of Deep Learning Pipelines and Architectures

From Linear Models to Modern Sequence Models: A Comparative Study for Image Classification

---

Course: CO3133 – Deep Learning and Its Applications
Instructor: Lê Thành Sách
Semester: 261
Group: G-03
Members: Nguyen Van An (2352013)
Vo Le Hai Dang (2352257)
Huynh Vuong Khang (2350011)

Ho Chi Minh City, October 2026

Contents

1 Problem and Data Description **2**

1.1 Problem formulation . . . 2

1.2 Dataset description . . . 2

1.3 Exploratory data analysis . . . 3

1.4 Data splitting . . . 5

1.5 Preprocessing . . . 5

2 Methodology **6**

2.1 Pipeline overview . . . 6

2.2 Baseline and main methods . . . 6

2.3 Training setup . . . 8

2.4 Experimental design . . . 9

3 Implementation Results **9**

3.1 Training behaviour . . . 9

3.2 Quantitative results . . . 11

3.3 Qualitative results . . . 11

3.4 Comparison and discussion . . . 14

3.5 Error analysis . . . 14

3.6 Limitations and conclusion . . . 15

AI usage disclosure **16**

1 Problem and Data Description

1.1 Problem formulation

Real-world motivation. Clothing recognition underlies product tagging, visual search and inventory automation in online retail. Fashion-MNIST is a controlled surrogate: small enough to train five architecture families to convergence on one consumer GPU, hard enough that none exceeds 93% test accuracy.

Input. One grayscale image $x \in \{0, \dots, 255\}^{1\times28\times28}$, given as a `uint8` tensor of shape $(N, 1, 28, 28)$.

Output and prediction unit. One of ten labels: T-shirt/top, Trouser, Pullover, Dress, Coat, Sandal, Shirt, Sneaker, Bag, Ankle boot. The prediction unit is one image; no fine-grained part annotations exist.

Mathematical formulation. A model f_θ maps an image to ten raw logits $z = f_\theta(x) \in \mathbb{R}^{10}$ and predicts $\hat{y} = \arg\max_k z_k$. Training minimises the cross-entropy loss

$$\mathcal{L}(\theta) = -\frac{1}{N}\sum_{i=1}^{N} \log \frac{\exp\left(z_{y_i}^{(i)}\right)}{\sum_{k=1}^{10} \exp\left(z_k^{(i)}\right)}, \tag{1}$$

computed directly on the logits: no softmax layer precedes `nn.CrossEntropyLoss`, which already applies the log-softmax. Optimisation is mini-batch Adam.

Main difficulties.

- *Silhouette ambiguity*: Shirt, T-shirt/top, Pullover and Coat are similar at 28×28, and almost all residual error concentrates there (Section 3.5).
- *No colour*: grayscale removes the strongest remaining cue.
- *Artificial ordering*: sequence models must linearise a 2-D image, imposing an order the image does not have.

1.2 Dataset description

Source, version and license. Fashion-MNIST, released in 2017 by Zalando Research as a drop-in replacement for MNIST, obtained through `torchvision.datasets.FashionMNIST` (torchvision 0.29.0) under the MIT license. torchvision verifies the MD5 checksum on download, so re-runs fetch byte-identical data.

Collection and labelling. 70,000 images sampled from Zalando article photographs, converted to 28×28 grayscale, centred and resized, and labelled with ten article categories by the dataset authors. The original photographs are not distributed.

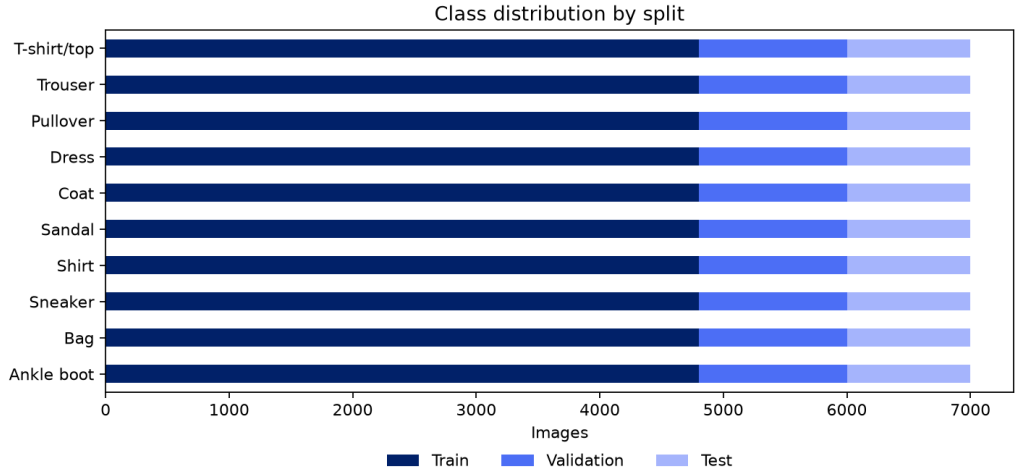
Sample counts and annotations. 70,000 labelled images, exactly 7,000 per class. Annotations are single class labels; there are no boxes, masks or secondary attributes.

Storage size and format. Four gzip-compressed IDX files, about 30 MB on disk. In memory the data occupies roughly 220 MB once cast to `float32` (70,000 × 784 × 4 bytes), plus the `uint8` source tensors.

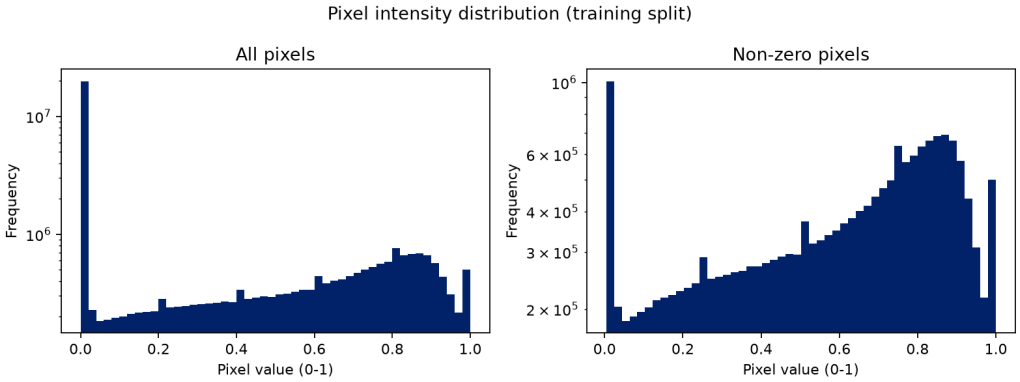
Bias and known limitations. The set is balanced by construction and therefore not representative of long-tailed retail catalogues. Images are single, centred objects on near-uniform dark backgrounds: the training images average 0.2863 with standard deviation 0.3532 on the $[0, 1]$ scale. Category boundaries are partly subjective (Shirt versus T-shirt/top versus Pullover versus Coat), which caps achievable accuracy independently of the model.

1.3 Exploratory data analysis

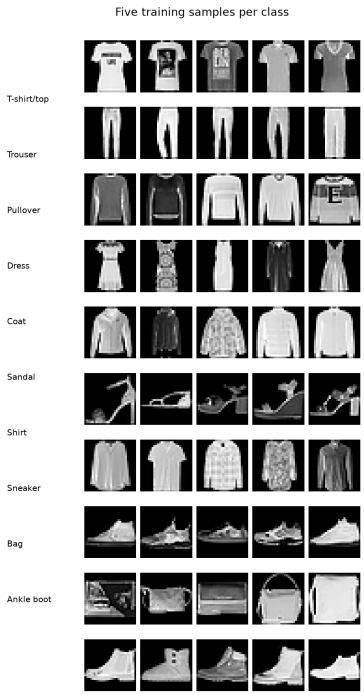
- **Class distribution and imbalance analysis.** The dataset is exactly balanced: after the custom validation split each class contributes 4,800 training, 1,200 validation and 1,000 test images, a maximum-to-minimum class ratio of 1.000. No class weighting or resampling is applied, and accuracy and macro-F1 differ by less than 0.1 point for every model (Table 7).
- **Input size.** All images are $28 \times 28 \times 1 = 784$ values; the input tensor is $(N, 1, 28, 28)$.
- **Representative samples.** Figure 1 shows five images per class, the class distribution and the pixel-value distribution; the full EDA, including non-zero-pixel and near-black background statistics, is in `assignment1/notebooks/eda.ipynb`.
- **Duplicates.** An FNV-1a 32-bit hash over the training images detected no exact duplicates requiring removal; the split sizes in Table 1 account for every image.



(a) Class distribution after the split



(b) Pixel-value distribution



(c) Representative samples

Figure 1: Exploratory analysis of the training split: the ten classes are exactly balanced, most pixels sit near the black background, and the sample grid shows the silhouette similarity of Shirt, T-shirt/top, Pullover and Coat that dominates the error analysis of Section 3.5.

1.4 Data splitting

Table 1: Data split used by all five models. The official test split is never used for training, validation, checkpoint selection or preprocessing statistics.

Split	Images	Per class	Source
Train	48,000	4,800	official train, 80% after stratified split
Validation	12,000	1,200	official train, 20% after stratified split
Test	10,000	1,000	official test split, untouched
Total	70,000	7,000	

- **Official or custom.** The official Fashion-MNIST train/test partition (60,000 / 10,000), with a custom validation set carved out of the official training set.
- **Seed and stratification.** `train_test_split(test_size=0.2, stratify=y, random_state=42)` on the 60,000 training images gives a stratified 48,000 / 12,000 split (Table 1); seed 42 is shared by every model.
- **Split unit.** One image. No entity spans images, so no group-level separation is required.
- **Leakage prevention.** The test split is touched once per model, after training and checkpoint selection; validation is drawn only from the training split; normalisation statistics come from the training split alone (Section 1.5).
- **Subset selection.** None. All 70,000 images are used, with no filtering, deduplication or relabelling.

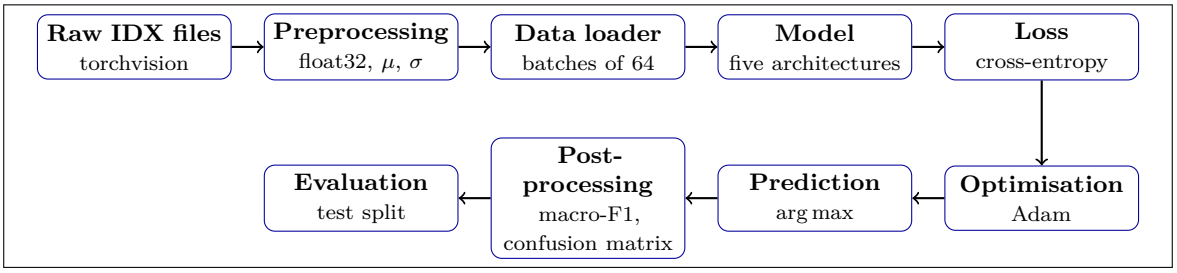
1.5 Preprocessing

- **Cleaning.** None required: every image is a well-formed 28×28 grayscale array with a valid label, and there are no missing values or invalid annotations.
- **Resizing.** None; all images share one resolution, so no interpolation is applied.
- **Normalisation.** Images are cast to `float32` and divided by 255 to give $[0, 1]$, then standardised with training-set mean $\mu = 0.2863$ and standard deviation $\sigma = 0.3532$: $x' = (x/255 - \mu)/\sigma$. The same μ, σ are applied to validation and test.
- **Input tensor shape after preprocessing.** $(N, 1, 28, 28)$ `float32`, labels $(N,)$ `int64`.
- **One example batch.** `X [64, 1, 28, 28] float32`, `y [64] int64`; at batch size 64 there are 750 training, 188 validation and 157 test batches.

2 Methodology

2.1 Pipeline overview

All five models follow the same pipeline:



2.2 Baseline and main methods

Table 2 lists the five mandatory models. All consume the same tensor, return raw logits of shape $(B, 10)$ and expose a parameter counter, so one training and evaluation routine drives all five.

Table 2: The five architecture families.

Model	Input representation	Model architecture	Params
Linear	flattened 784-vector	`Flatten` → `Linear`(784, 10)	7,850
MLP	flattened 784-vector	784 → 256 → 128 → 10, ReLU	235,146
CNN	28×28 image, kept 2-D	two blocks of [Conv 3×3, pad 1 → ReLU → MaxPool 2×2] with 32 then 64 channels, then an MLP head 3,136 → 128 → 256 → 10	455,946
GRU	28 row tokens of 28 values	explicit `GRUCell` loop over $T = 28$ steps, hidden size 128, one layer; final hidden state → `Linear`(128, 10)	61,962
Transformer	28 row tokens of 28 values	`Linear`(28, 64) token projection, fixed sinusoidal positional encoding, 2 encoder layers (4 heads, FFN 128, dropout 0.1), mean pooling → `Linear`(64, 10)	69,450

Linear / softmax classifier. Each image is flattened to 784 values and mapped by one linear layer to ten logits. It has no notion of neighbouring pixels and is the reference point for spatial inductive bias. Strength: 7,850 parameters, about 10 s to train. Weakness: it cannot represent any spatial interaction.

Multilayer perceptron. 784 → 256 → 128 → 10 with ReLU activations, satisfying the hidden-layer requirement. The activation makes the composition non-linear, allowing pixel combinations to be learned; hidden layers permit non-linear decision boundaries the linear model cannot form. It still flattens the image, so locality must be relearned by separate weights at every position. Regularisation: none (no dropout, no weight decay); the resulting gap is quantified in Section 3.1.

Convolutional network. Designed here, trained from scratch, no pretrained backbone. Two blocks of [3×3 convolution, padding 1, stride 1 → ReLU → 2×2 max pooling] with 32 then 64 channels reduce 28×28 to 7×7 and produce 64 feature maps, one response plane per learned detector. Each convolution shares one kernel across all positions, so a local pattern is detected

wherever it occurs with far fewer weights than a dense layer; pooling halves the spatial resolution, reducing computation and adding translation tolerance. The 3,136 flattened values feed an MLP head whose input width is measured by one dummy forward pass, so the convolutional layout can change without editing the head.

GRU. Each image row is one timestep: $T = H = 28$ steps consuming a 28-dimensional row vector, so sequence length and input size are distinct quantities. The hidden state $h_t \in \mathbb{R}^{128}$ summarises the rows read so far, $h_t = \mathrm{GRU}(x_t, h_{t-1})$ from $h_0 = 0$, and h_T is classified. The recurrence is an explicit single-step `GRUCell` loop, which also serves `LSTMCell` and `RNNCell` variants; the LSTM variant exists but is not reported. Strength: vertical order is modelled explicitly with few parameters. Weakness: a row sequence has no 2-D neighbourhood operator.

Transformer. Rows are used as tokens. A linear projection maps each 28-dimensional row to $d_{\text{model}} = 64$; a fixed sinusoidal positional encoding is added so row order is distinguishable (a non-trainable buffer, so no extra parameters); two encoder layers apply multi-head self-attention, each head computing $\mathrm{softmax}(QK^\top/\sqrt{d_k})V$ over all 28 tokens with no causal mask; mean pooling over token outputs gives one vector per image for the linear head. Attention inputs are the projected, position-encoded tokens; its output is a same-length sequence of context vectors; every token attends to every other. Strength: unrestricted token-to-token interaction. Weakness, shared with the GRU: the 1-D row order is an artefact of the linearisation.

Self-implemented and library components. All five architectures, the training loop and the evaluation logic are self-implemented; libraries supply dataset access, numerical primitives and metrics (Table 3).

Table 3: Provenance of each component. The five mandatory architectures are self-implemented; libraries provide primitives and metrics only.

Component	Origin
Linear, MLP, CNN, GRU, Transformer	Self-implemented in `assignment1/src/models/`. The CNN topology was designed here; no pretrained backbone or pretrained weights are used.
Recurrence: timestep loop and hidden-state handling	Self-implemented; `nn.GRUCell`, `nn.LSTMCell` and `nn.RNNCell` provide a single step only.
Token projection, sinusoidal positional encoding, pooling, classification heads	Self-implemented; `nn.TransformerEncoder` and `nn.TransformerEncoderLayer` provide multi-head self-attention.
Training loop, validation, checkpoint selection, metric aggregation, all figures	Self-implemented in `assignment1/notebooks/main.ipynb`, `assignment1/src/training_loop.py` and `utils/experiment_utils.py`.
Convolution, pooling, optimiser, loss, autograd	PyTorch (`torch.nn`, `torch.optim`).
Dataset download, IDX parsing, checksum verification	torchvision (`torchvision.datasets`).
Stratified train/validation split	scikit-learn (`train_test_split`).
Accuracy, macro-F1, precision/recall/F1, confusion matrix	scikit-learn (`metrics`).
Figure rendering, results tables, array handling	Matplotlib, pandas, NumPy.

2.3 Training setup

Table 4: Training configuration, identical for all five models.

Item	Value
Optimiser	Adam
Learning rate	1×10^{-3}, constant (no scheduler)
Batch size	64
Epochs	10, fixed budget for every model
Loss	`nn.CrossEntropyLoss` on raw logits
Gradient clipping	global norm clipped to 1.0, all models
Regularisation	none, except dropout 0.1 inside the Transformer encoder
Early stopping	none
Checkpoint criterion	highest validation macro-F1, one checkpoint per model
Evaluation checkpoint	the selected checkpoint, not the final epoch
Mixed precision	not used
Seed	42 (NumPy and PyTorch)
Hardware	NVIDIA GeForce RTX 3050 Laptop GPU (4 GB), CUDA 13.0
Software	Python 3.14.7, PyTorch 2.14.0, torchvision 0.29.0, NumPy 2.5.3, pandas 3.0.6, Matplotlib 3.11.2, scikit-learn 1.8.0
Hyperparameter selection	fixed a priori, not tuned per model

Timing definition. Training time covers all ten epochs including the per-epoch validation pass and checkpoint writing (mean per-epoch cost: Linear 1.0 s, MLP 1.3 s, CNN 2.1 s, GRU 5.3 s, Transformer 3.2 s). Inference time is one forward pass over the 10,000 test images after a two-batch warm-up, synchronised on the GPU, excluding loss and metric computation.

Determinism. Seed 42 fixes the split, the preprocessing statistics, the DataLoader order and the initial weights. Deterministic CUDA algorithms are not enabled, so rerunning the same script reproduces the protocol exactly and the metrics only approximately. This affects training only; inference from a fixed checkpoint is deterministic.

2.4 Experimental design

Table 5: Design of the main comparison.

Item	Statement
Hypotheses	**H1.** A model with spatial inductive bias (CNN) outperforms models that flatten the image (Linear, MLP). **H2.** Sequence models preserving one axis of order (GRU, Transformer) beat the flattening baselines without matching the CNN. **H3.** Accuracy is not monotone in parameter count.
Changed factor	The architecture family only.
Fixed factors	Dataset, split (48,000/12,000/10,000, seed 42), preprocessing, batch size 64, 10 epochs, Adam at 10^{-3}, gradient clipping, checkpoint criterion, evaluation protocol.
Metrics	Test accuracy and test macro-F1 (primary); parameter count, training time and inference time (secondary); curves, confusion matrices and prediction examples as qualitative evidence.
Conclusion criteria	A difference counts as established only if it exceeds one point of test accuracy and holds for both primary metrics; smaller differences are unresolved, since one seed gives no variance estimate.

3 Implementation Results

3.1 Training behaviour

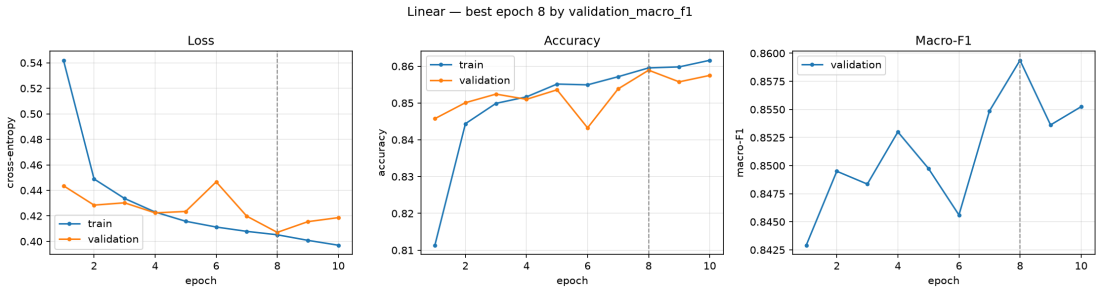
Training and validation loss, accuracy and validation macro-F1 are shown in Figure 2; Table 6 summarises the final and selected epochs.

Table 6: Training behaviour. "Gap" is final-epoch training accuracy minus final-epoch validation accuracy. "Δ best–final" is the selected epoch's validation macro-F1 minus the final epoch's.

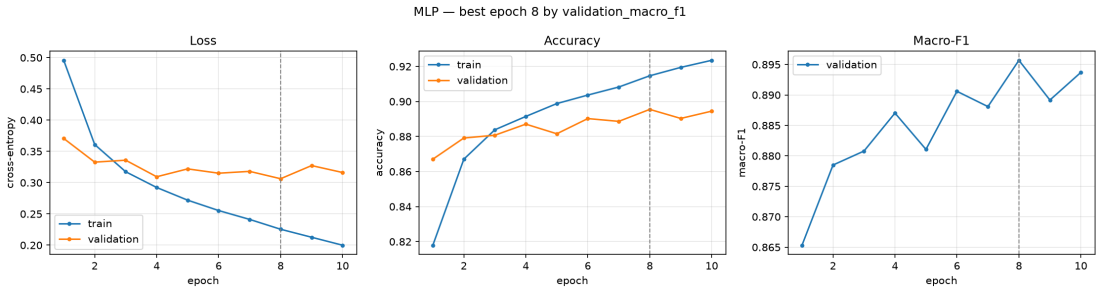
Model	Final train acc	Final val acc	Gap (pts)	Δ best–final (pts)	Best epoch
Linear	86.16%	85.75%	0.41	0.41	8
MLP	92.35%	89.45%	2.90	0.20	8
CNN	97.48%	92.49%	4.99	0.26	7
GRU	91.95%	90.08%	1.87	0.07	8
Transformer	90.20%	89.06%	1.14	0.59	8

Underfitting and overfitting. Training accuracy exceeds 86% for all five, so none underfits. The MLP and CNN show early overfitting (gaps of 2.90 and 4.99 points), having the most capacity and no explicit regularisation. The GRU and Transformer stay within 1.9 and 1.1 points, and the linear model has the smallest gap because it cannot overfit spatially.

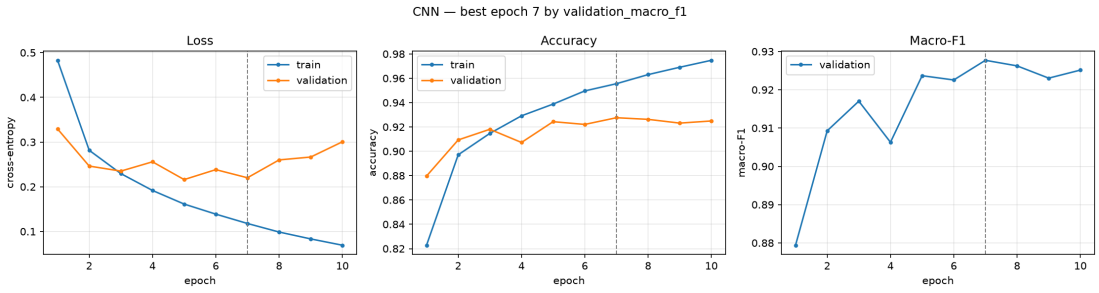
The selected checkpoint. Four models peaked at epoch 8 of 10 and the CNN at epoch 7; validation performance then flattened or fell slightly (Table 6). Selecting the final epoch would have cost 0.07 to 0.59 points of validation macro-F1, most for the Transformer. This is consistent with a constant learning rate and no schedule or early stopping: training loss keeps falling while validation performance oscillates. All reported test numbers come from the selected checkpoint, reloaded before the test pass.



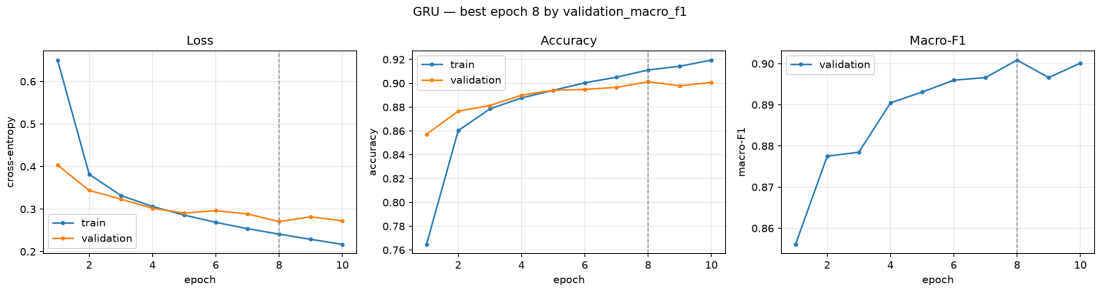
(a) Linear



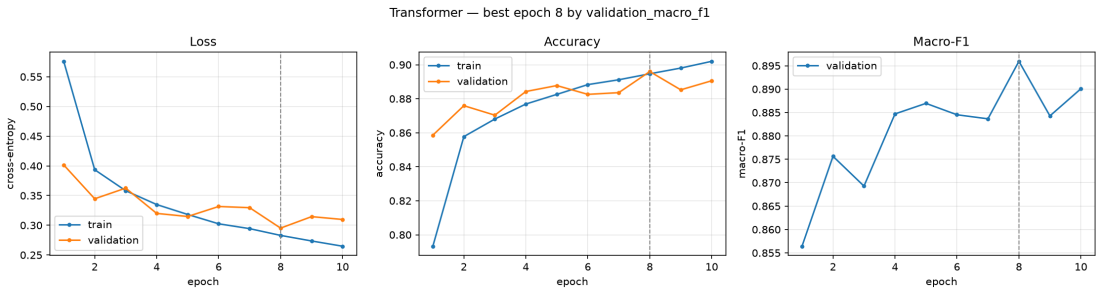
(b) MLP



(c) CNN



(d) GRU



(e) Transformer

Figure 2: Training and validation curves. The dashed vertical line marks the selected epoch (highest validation macro-F1).

Implementation issues and fixes.

- *dtype mismatch*: `uint8` inputs to `nn.Linear` raise a Byte/Float error; the shared preprocessing casts to `float32`.
- *Preprocessing leakage*: computing μ, σ over all data would leak test statistics; they are computed on the training split only.
- *Checkpoint selection*: reporting final-epoch weights underreported validation macro-F1 by 0.07–0.59 points; the loop restores the best-validation state before evaluation.
- *torchvision layout*: archives are read from `data/FashionMNIST/raw/`, so the loader points torchvision at the repository `data/` directory and lets it verify and extract.

3.2 Quantitative results

Table 7: Main results on the official 10,000-image test split. Validation columns are at the selected epoch; test columns are from that checkpoint. Training time covers 10 epochs including per-epoch validation.

Model	Params	Best epoch	Val acc	Val macro-F1	Test acc	Test macro-F1	Train (s)	Infer (ms/img)
Linear	7,850	8	85.89%	85.94%	84.10%	84.15%	9.6	0.005
MLP	235,146	8	89.55%	89.57%	87.89%	87.89%	12.8	0.006
CNN	455,946	7	92.76%	92.77%	**91.79%**	**91.79%**	20.5	0.010
GRU	61,962	8	90.13%	90.09%	88.93%	88.85%	52.5	0.025
Transformer	69,450	8	89.60%	89.59%	88.71%	88.69%	32.5	0.012

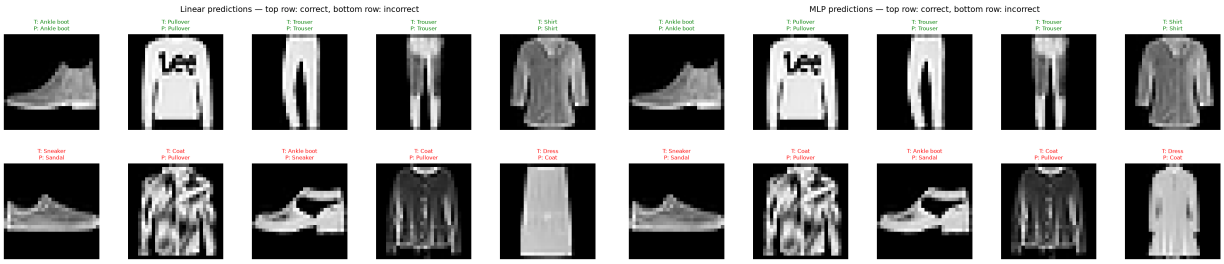
Accuracy and macro-F1 agree within 0.1 point for every model (Table 7), the expected consequence of an exactly balanced test split, so no model trades one metric for the other. Test accuracy is 0.9 to 1.8 points below validation accuracy: the validation split is slightly optimistic but ranks the models as the test split does.

3.3 Qualitative results

Table 8: Per-class F1 on the test split. Shirt is the hardest class for every model; the CNN improves it by 16 points over the linear model.

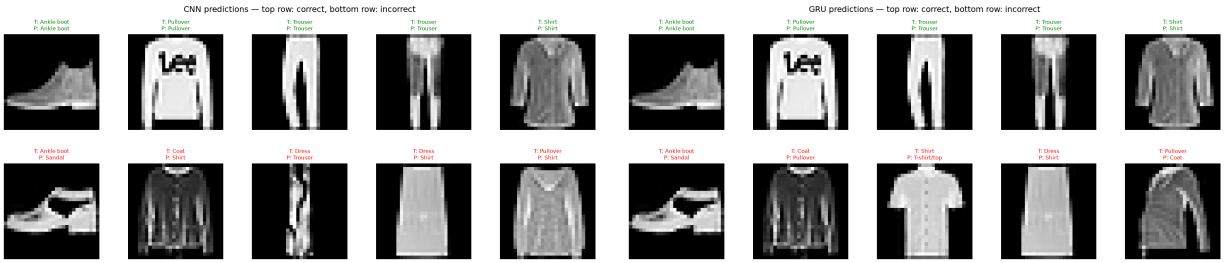
Class	Linear	MLP	CNN	GRU	Transformer
T-shirt/top	0.791	0.826	**0.867**	0.839	0.824
Trouser	0.966	0.981	**0.986**	0.978	0.981
Pullover	0.726	0.781	**0.879**	0.807	0.803
Dress	0.849	0.884	**0.918**	0.890	0.896
Coat	0.748	0.803	**0.878**	0.814	0.812
Sandal	0.931	0.965	**0.977**	0.972	0.968
Shirt	0.601	0.682	**0.765**	0.712	0.710
Sneaker	0.922	0.944	**0.956**	0.949	0.946
Bag	0.935	0.964	**0.987**	0.968	0.974
Ankle boot	0.946	0.959	**0.967**	0.957	0.955

Figure 3 shows five correct and five incorrect predictions per model and Figure 4 the full confusion matrices. The easiest classes for every model are Trouser, Bag and Sandal, where silhouette is distinctive; difficult-but-correct cases are usually unusual poses of those classes. Failures concentrate on upper-body garments (Section 3.5).



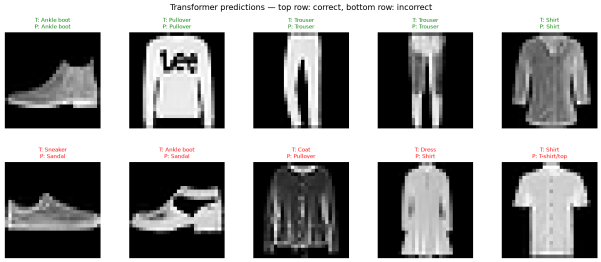
(a) Linear

(b) MLP



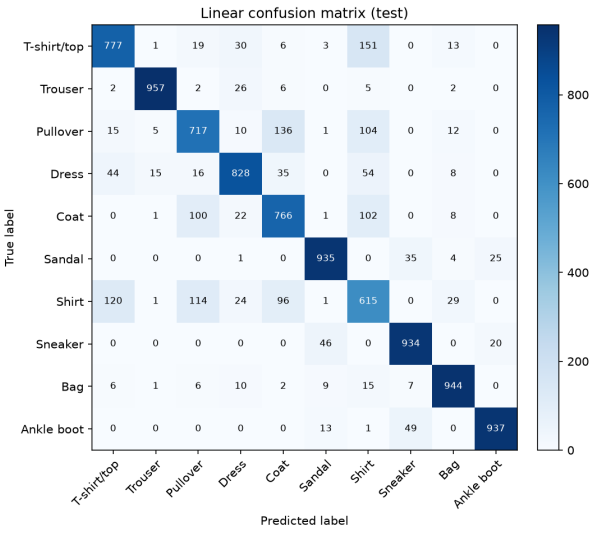
(c) CNN

(d) GRU

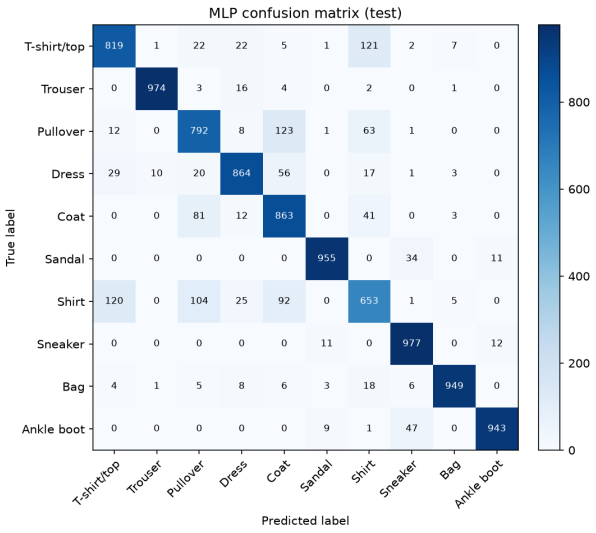


(e) Transformer

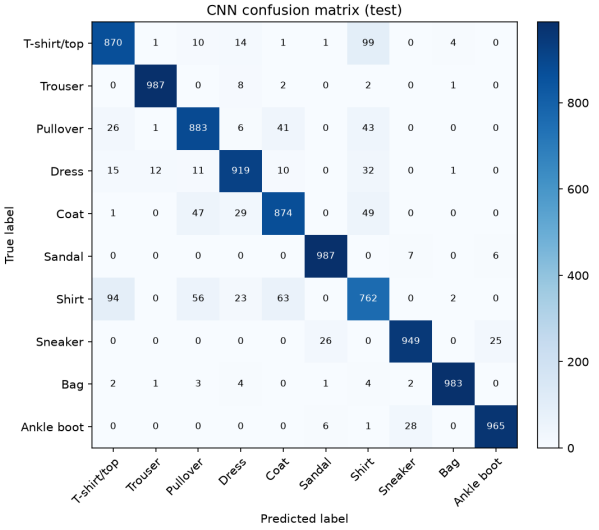
Figure 3: Test predictions per model. Top row of each panel: correctly classified examples; bottom row: misclassified examples, with true (T) and predicted (P) labels.



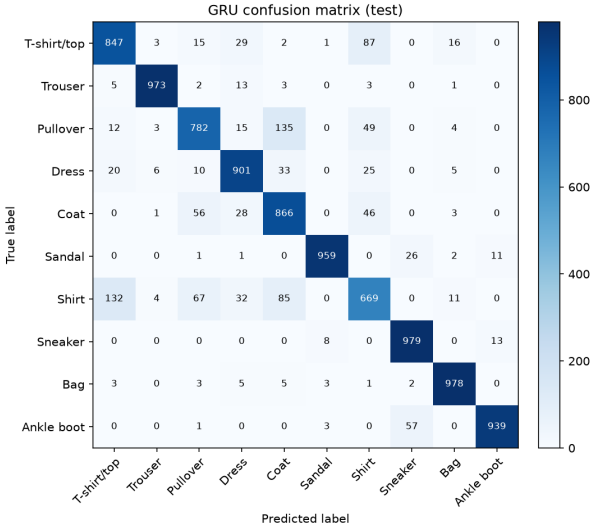
(a) Linear



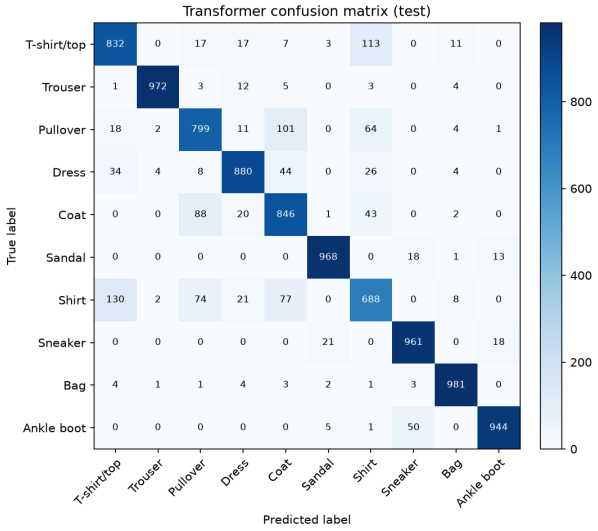
(b) MLP



(c) CNN



(d) GRU



(e) Transformer

Figure 4: Confusion matrices on the 10,000-image test split (true label by row, predicted by column).

3.4 Comparison and discussion

Which model is better, and by how much. The CNN is best on both primary metrics: 91.79% test accuracy and 91.79% test macro-F1. It leads the GRU by 2.86 points, the Transformer by 3.08, the MLP by 3.90 and the linear classifier by 7.69 – above the one-point threshold of Table 5 and consistent across both metrics and across validation and test.

H1 (spatial inductive bias) is supported. Both unflattened models beat every flattened model. The ordering Linear < MLP < CNN matches increasing spatial structure: no locality, then learned but unshared locality, then weight-shared convolutions. The CNN also has the best F1 on all ten classes (Table 8), so its advantage is not class-specific.

H2 (sequence models beat flattening) is partially supported. Both sequence models beat the linear baseline clearly, using only row order. Against the MLP, the GRU leads by 1.04 points of accuracy but 0.96 of macro-F1, and the Transformer by 0.82 and 0.80 – below the one-point rule of Table 5, so those two margins remain unresolved. Neither approaches the CNN, indicating that vertical order is a weaker prior than 2-D locality for this task.

H3 (accuracy is not monotone in parameters) is supported. The MLP has 235,146 parameters and reaches 87.89%, while the GRU has 61,962 – about a quarter – and reaches 88.93%. The CNN's 455,946 parameters do give the best accuracy, but the ordering of the middle three is not explained by capacity. Capacity is not the binding constraint; representation is.

Trade-offs. Parameter count, accuracy and speed favour different models. The GRU is the most parameter-efficient (88.93% for 61,962 parameters) but the slowest to train, 52.5 s or about 4.1× the MLP, because each batch runs 28 sequential cell steps that cannot be parallelised; it is also slowest at inference (0.025 ms/sample, 2.5× the CNN). The Transformer trains 1.6× faster than the GRU (32.5 s) and infers about 2.1× faster for 0.22 points less accuracy. The CNN gives the best accuracy at mid-range cost (20.5 s, 0.010 ms/sample). The linear model trains 5.5× faster than the GRU with 58× fewer parameters than the CNN for 7.69 points less accuracy.

What cannot yet be concluded. With one seed, GRU > Transformer (0.22 points) and MLP < Transformer (0.82 points) are unresolved differences. The CNN's advantage cannot be attributed to a single component (depth, pooling, kernel size, head width) without an architectural ablation, and the rankings are not established beyond this 10-epoch, untuned budget.

3.5 Error analysis

Error counts on the 10,000-image test split: Linear 1,590, MLP 1,211, Transformer 1,129, GRU 1,107, CNN 821. Their structure is nearly identical across models (Table 9); Table 10 lists the largest individual confusions.

Error group 1: upper-body clothing (60–65%). T-shirt/top, Pullover, Coat and Shirt are mutually confused by every model, and Shirt is the hardest class for all five (F1 0.601 linear, 0.682 MLP, 0.712 GRU, 0.710 Transformer, 0.765 CNN); examples are in the bottom row of Figure 3. Cause: at 28×28 the discriminator is a few pixels around shoulders and neckline, and grayscale removes the fabric colour and texture that separate these classes in the original photographs. Improvements: light augmentation (small translations, horizontal flips), higher input resolution, or a deeper CNN / spatial attention to widen the receptive field around collars.

Table 9: Where the errors are. Percentages are of that model's own error total. Confusions are partitioned by whether true and predicted classes belong to the same visual group (upper-body: T-shirt/top, Pullover, Coat, Shirt; footwear: Sandal, Sneaker, Ankle boot; trouser/dress; bag).

Error group	Linear	MLP	CNN	GRU	Transformer
Total errors	1,590	1,211	821	1,107	1,129
Upper-body ↔ upper-body	60.6%	64.7%	64.6%	62.0%	**64.8%**
Footwear ↔ footwear	11.8%	10.2%	11.9%	10.7%	11.1%
Trouser ↔ Dress	2.6%	2.1%	2.4%	1.7%	1.4%
Cross-group	25.0%	22.9%	21.1%	25.7%	22.7%

Table 10: Largest individual confusions (true → predicted, count) on the test split.

Model	Top confusions
Linear	T-shirt/top→Shirt 151; Pullover→Coat 136; Shirt→T-shirt/top 120; Shirt→Pullover 114; Pullover→Shirt 104
MLP	Pullover→Coat 123; T-shirt/top→Shirt 121; Shirt→T-shirt/top 120; Shirt→Pullover 104; Shirt→Coat 92
CNN	T-shirt/top→Shirt 99; Shirt→T-shirt/top 94; Shirt→Coat 63; Shirt→Pullover 56; Coat→Shirt 49
GRU	Pullover→Coat 135; Shirt→T-shirt/top 132; T-shirt/top→Shirt 87; Shirt→Coat 85; Shirt→Pullover 67
Transformer	Shirt→T-shirt/top 130; T-shirt/top→Shirt 113; Pullover→Coat 101; Coat→Pullover 88; Shirt→Coat 77

Error group 2: footwear (10–12%). Sandal, Sneaker and Ankle boot are confused mainly with each other; the largest pair is Sneaker↔Ankle boot for the MLP (59), GRU (70) and Transformer (68), and Sandal↔Sneaker for the linear model (81). Cause: the three classes share one footprint and differ in upper height and strap detail, which a flattening model cannot encode and a 7×7 feature map begins to lose. Improvements: drop the second pooling stage, or add a branch at the higher-resolution feature map.

Error group 3: cross-group confusions (21–26%). Most remaining errors predict another garment family entirely, typically a Coat or Pullover called a Dress (long garment, ambiguous waist) or a Bag called a T-shirt/top (both compact dark blobs).

The CNN's residual errors. The CNN's errors have the same structure – 64.6% upper-body, largest confusion T-shirt/top→Shirt (99) – but at 821 errors it removes about half of the linear model's 1,590. The ambiguity is reduced, not eliminated, so the remaining 8.2% error is largely dataset-intrinsic rather than an architectural deficiency.

3.6 Limitations and conclusion

Limitations. Data: one dataset, grayscale, exactly balanced, single object per image, so the error analysis says nothing about minority-class behaviour in a long-tailed catalogue. Method: one seed (42) with no variance estimate or confidence interval; a fixed 10-epoch budget with no schedule and no early stopping, so four models stopped at epoch 8 and the CNN at epoch 7; hyperparameters fixed a priori rather than searched, with one learning rate for all five families; no architectural ablation; no augmentation in the main pipeline; no dropout or weight decay in the MLP and CNN heads, which explains their larger gaps; both sequence models linearise the image into rows, so no patch tokenisation was tested.

Conclusion. Under one shared split, preprocessing and protocol, the self-designed CNN is the best of the five required architecture families on Fashion-MNIST, at 91.79% test accuracy and 91.79% test macro-F1, ahead of the GRU (88.93% / 88.85%), the Transformer (88.71% / 88.69%), the MLP (87.89%) and the linear classifier (84.10%). The ranking follows inductive bias rather than capacity: the most parameter-efficient model is not the largest, and the flattening models lose locality that dense capacity does not recover. Spatial structure, not sequence order and not parameter count, is the dominant factor on this task, and the residual error is concentrated in one visually ambiguous class family rather than spread across the label set.

AI usage disclosure

In accordance with the course AI-use policy, the per-tool log is in `assignment1/AI_USAGE.md` (root: `AI_USAGE.md`). Summary for this report:

- **Tools used.** ChatGPT-4 (initial page boilerplate), Gemini (EDA plotting and augmentation examples), and an `opencode` CLI agent (module refactoring, model documentation, notebook scaffolding, experiment plumbing).
- **Members involved.** Vo Le Hai Dang (page boilerplate), Huynh Vuong Khang (EDA visualisation, augmentation examples), Nguyen Van An (data pipeline, models, experiments).
- **AI contribution.** Loading code refactored into `utils/dataloader.py`; documentation and formatting of the five model modules; the explicit recurrent-cell scaffold and the row-token Transformer; the five-model training/evaluation notebook including checkpointing, metrics and figure generation. The typesetting of this report was AI-assisted; the technical content and all figures come from the group's own runs.
- **Verification.** Every reported number was produced by executing `assignment1/notebooks/main.ipynb` on the group's hardware; the reported run corresponds to tag `a1-m2-final` (commit `a783829`) in `github.com/DangXiMi/CO3133-DeepLearning`; the committed artifacts (`results.csv`, `results.json`, per-epoch histories, checkpoints, figures) were regenerated from that run, and histories were cross-checked against the selected checkpoints. Members reviewed and edited the code and are responsible for explaining every component.
- **Responsible member.** Nguyen Van An (data pipeline, models, experiments); Vo Le Hai Dang (pages, repository coordination); Huynh Vuong Khang (EDA, visualisation).